Technical Report - Book Metadata and Recommendation API

Changwei Li

GitHub Repository: https://github.com/qt0713/Web-cwk

API documentation: API_DOCUMENTATION.pdf/qt0713/Web-cwk

1. Project Overview

This project implements a RESTful Book Metadata and Recommendation API for backend coursework. It includes authenticated CRUD, recommendation retrieval, analytics endpoints, and reproducible public-dataset import.

Project objectives

- Deliver a runnable SQL-backed API with authenticated CRUD.
- Extend baseline features with recommendation, analytics, and reproducible import.

2. Technology Choices and Rationale

Programming language and framework

Python with FastAPI was selected for rapid endpoint development, validation, and automatic OpenAPI generation.

Database

The project uses two SQL targets:
- PostgreSQL as the production-oriented target.
- SQLite as a local development and testing option for fast setup and demonstrability.

ORM and data access

SQLAlchemy was chosen for relational modelling and cross-database compatibility.

Testing stack

Pytest with FastAPI TestClient was used for endpoint-level integration tests of real request/response behavior.

3. Architecture and Design Decisions

API style

The application uses RESTful resource-oriented routing under `/api/v1`, with separate route groups for authentication, books, and analytics.

Core data model

Primary entity: `Book`

Key fields:
- title
- author

- genre
- average_rating
- ratings_count

Recommendation strategy
A lightweight heuristic is used:
1. Prioritize books in the same genre as the seed book.
2. Rank by average_rating, then ratings_count.
3. If not enough items exist, fill with globally top-ranked books.

This approach is interpretable and suitable for coursework constraints.

Security model
JWT-based authentication is implemented via:
- register endpoint,
- login endpoint,
- bearer-token protection on book and analytics endpoints.

Error handling
The API uses standard status codes and explicit errors (200/201/204, 401, 404, 409, 422).

4. Testing Approach and Results

Scope
Integration tests cover health, authenticated access, CRUD behavior, recommendation output, and analytics accessibility.

Method
Pytest + FastAPI TestClient with an isolated SQLite test database per run.

Result summary
Current automated test status: all tests pass, confirming that core API flows remain functional after data-import and analytics enhancements.

5. Public Dataset Integration

Dataset source
- Kaggle dataset: arashnic/book-recommendation-dataset
- Files used: Books.csv and Ratings.csv

Import implementation
A dedicated import script ingests metadata, handles CSV encoding/format inconsistencies (including ZIP edge cases), and merges rating statistics from Ratings.csv via ISBN mapping.

Executed import record

- Command: python scripts/import_kaggle_books.py --csv-path .\Books.csv --ratings-csv-path .\Ratings.csv --limit 1000 --replace
- Result: inserted 1000, skipped 0.
- Verification: database book count = 1000.

6. Analytics Validation

After import, analytics endpoints returned HTTP 200.

Observed behavior:
- /api/v1/analytics/genres and /api/v1/analytics/ratings both returned consistent non-empty outputs.

Rating scale note

The imported ratings source uses a 0-10 scale, so analytics averages can exceed 5.0. This reflects source-scale preservation, not a normalization defect.

7. Challenges and Resolutions

Challenge 1: dataset ingestion reliability

Real-world CSV files included encoding and packaging irregularities; parser robustness was improved with fallback extraction logic.

Challenge 2: balancing complexity with demonstrability

A fully personalized recommender was out of scope, so a transparent heuristic model was implemented for correctness and explainability.

8. Limitations

- Recommendation logic is heuristic and not user-personalized.
- Genre values are coarse due to source-field constraints.
- Ratings remain source-scale dependent (0-10).
- Advanced production controls (for example full observability and RBAC) are not yet implemented.

9. Future Improvements

- Add optional rating normalization strategy (0-10 to 0-5).
- Add user profiles and interaction history for personalized recommendation.
- Add richer pagination/filter/sorting and CI quality gates.
- Harden the PostgreSQL production profile with monitoring.

10. Deployment and Operational Status

Deployment target and runtime stack

Deployment target: Tencent Cloud CVM (Ubuntu 22.04), with:
- Uvicorn process managed by systemd (bookapi.service).

- Nginx reverse proxy on port 80 forwarding to 127.0.0.1:8000.
- SQLite-backed runtime for coursework demonstration.

Deployment validation performed on server
Server-side checks completed successfully:
- bookapi.service running (active (running)).
- curl http://127.0.0.1:8000/health -> HTTP 200.
- curl http://127.0.0.1/health -> HTTP 200.
- curl http://127.0.0.1/docs -> HTTP 200.

External connectivity issue and mitigation
External access timed out from some client networks despite healthy in-host checks.

Mitigation for oral-demo accessibility:
- A temporary reverse tunnel endpoint was established from the server to a public HTTPS URL.
- Public access was validated through /health and /docs.

Operational note for examiners
If a temporary tunnel URL is used, it may rotate between sessions and requires an active tunnel process. Core reproducibility remains verifiable through repository setup and server-side checks.

11. Generative AI Declaration

Tools used
- GitHub Copilot
- ChatGPT

Purpose and level of use
- architecture planning,
- option exploration,
- debugging support,
- importer hardening,
- documentation refinement.

Use was systematic: each major task followed objective -> options -> decision -> validation.

Examples of alternative exploration
- Authentication: API-key-only vs JWT token flow.
- Deployment: platform-managed hosting vs CVM runtime + fallback tunnel path.
- Data ingestion: strict parser vs resilient fallback parser pipeline.
- Example image is shown below：

For authentication, the assessment requires demonstrating modern practices. Compare: API Key only vs JWT login flow vs Swagger-compatible OAuth2 password flow. Which is most appropriate?

Here's a comparison aligned with academic project goals: clarity, defensibility, and matching "modern practices" expectations.

1. API Key Only

- ✅ Simple to implement (`X-API-Key` header).
- ❌ No user sessions; hard to show role-based access (admin vs regular user).
- ❌ Feels outdated for web APIs where user identity matters.

2. JWT Login Flow

- ✅ Stateless; scalable and widely adopted.
- ✅ Clear story: login → token → protected routes with roles.
- ✅ Works smoothly with frontends and mobile clients.
- ⚠️ Requires careful token expiration and refresh logic.

3. OAuth2 Password Flow (Swagger-compatible)

- ✅ Strongest "industry standard" signal.
- ✅ Swagger UI integrates nicely for examiners to test.
- ❌ More setup (scopes, token revocation, client credentials).
- ❌ Overkill unless the brief explicitly asks for OAuth2.

Recommendation: JWT Login Flow

It balances effort and impact:

- Demonstrates **stateless auth**, **protected routes**, and **user roles** cleanly.
- Easier to explain and defend orally than OAuth2.
- Avoids the "too minimal" impression of API keys.

Choices were evaluated against coursework criteria, explainability, and runtime reliability.

Verification and control

- AI-generated suggestions were reviewed before adoption.
- Functional correctness was validated through endpoint checks and automated tests.
- Security, importer, and analytics decisions were manually verified and adjusted.

Evidence-backed workflow quality

GenAI use supported design trade-offs and contingency planning.

Supplementary evidence of conversation excerpts

- Selected excerpts are provided in docs/GENAI_EXCERPTS.md
- Raw exported snippets are provided in docs/GENAI_EXPORTED_LOG_SNIPPETS.md
- Evidence window for submission packaging: 2026-04-10 to 2026-04-20.

Responsible use statement

Generative AI was used as an assistive tool, not as an unverified code source. Final decisions and submission outputs were student-controlled.

12. References

Book Recommendation Dataset. *www.kaggle.com*. [Online]. Available from: https://www.kaggle.com/datasets/arashnic/book-recommendation-dataset.

FastAPI 2023. FastAPI. *fastapi.tiangolo.com*. [Online]. Available from: https://fastapi.tiangolo.com/.

Appendix

1. Scope
   This appendix summarises the implemented database schema used by the API runtime and tests.

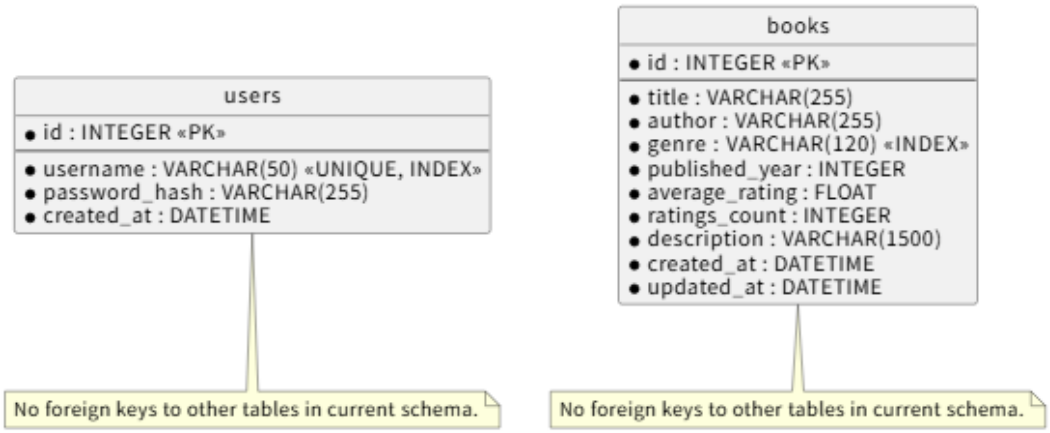
2. Core Tables

- users: id, email, hashed_password, is_active, created_at
- books: id, title, author, genre, published_year, average_rating, ratings_count, description, created_at, updated_at

3. Constraints and Integrity

- Primary keys on users.id and books.id
- Unique constraint on users.email
- Non-null constraints on required fields (for example title/author where applicable)
- Timestamp columns used for auditability

4. ORM Mapping Source
   Schema definitions are implemented via SQLAlchemy models in:

   user.py, book.py, base.py, session.py

5. The chart below shows the structure of the database